

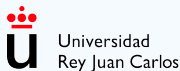
Big Data Processing II

Tema 3: Optimización y Escalabilidad de Pipelines Deportivos

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Conocer **buenas prácticas** de diseño de pipelines de análisis de datos
- Identificar técnicas de **optimización y escalabilidad** en Kafka y Spark
- Reconocer opciones de infraestructura en producción **on-premises y cloud**



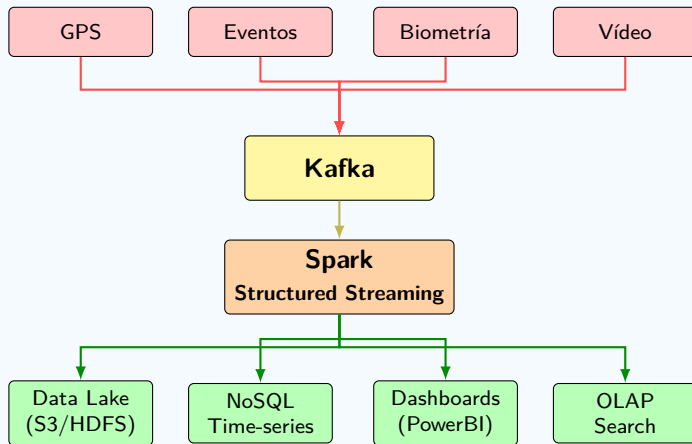
- 1 **Introducción**
- 2 **Buenas prácticas**
- 3 **Puesta en producción**



- 1 **Introducción**
- 2 Buenas prácticas
- 3 Puesta en producción



- **Múltiples fuentes de datos en vivo** → Como tracking, sensores, eventos y vídeo
- **Kafka para mensajería distribuida** → Bus de ingestión y retención de eventos
- **Spark para análisis de datos** → Tanto en batch como en streaming
- **Múltiples salidas operativas** → Como data lakes, BBDD y dashboards





Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

Fuente: <https://www.scaler.com/topics/spark-streaming-with-kafka/>



- **Rapidez en el análisis**

- Respuesta en tiempo real durante el partido
- Alertas y apoyo a decisiones en vivo

- **Capacidad y fiabilidad del sistema**

- Procesamiento de grandes volúmenes de datos deportivos
- Integración de dispositivos tracking, eventos y sensores
- Funcionamiento estable en momentos críticos de competición

- **Los objetivos marcan la arquitectura y los recursos**

- SLA/SLO claros para guiar decisiones
- Latencia objetivo y disponibilidad esperada



- **El cuello de botella se desplaza entre componentes**

- Optimizar un componente no garantiza mejorar todo el pipeline
- Al mejorar una parte la limitación suele aparecer en otra
- La optimización debe evaluarse a nivel global

- **La monitorización es imprescindible**

- Comparar métricas antes y después de cada cambio
- Monitorizar retrasos en Kafka y tiempos en Spark
- Vigilar uso de recursos como disco, red y memoria



- **Kafka escala mejor cuando**

- Hay suficientes particiones para paralelizar producción y consumo
- Se añaden brokers para repartir carga y almacenamiento
- La infraestructura ofrece buena red y discos rápidos

- **Spark escala mejor cuando**

- Hay suficiente particionado para el paralelismo de tareas
- Se asignan executors y memoria adecuados
- Se reducen los shuffles innecesarios



- 1 Introducción
- 2 Buenas prácticas**
- 3 Puesta en producción



- **Gobierno y estabilidad del dato**

- Mantener un esquema estable en pipelines críticos
- Definir reglas de evolución para evitar cambios incompatibles

- **Organización del almacenamiento**

- Elegir un particionado coherente con las consultas más frecuentes
- Compactar periódicamente para mejorar el rendimiento
- Un buen diseño mejora el rendimiento de lectura y escritura

- **Diseño según coste, latencia y tipo de procesamiento**

- En streaming se priorizan baja latencia y estabilidad operativa
- En batch se priorizan throughput y coste por volumen procesado
- Separar datos recientes (calientes) de históricos (fríos) ayuda a equilibrar ambos



- **Un buen particionado reparte la carga del pipeline**
 - Nº de particiones en Kafka = paralelismo máximo de consumo
 - En Spark cada partición se procesa como una tarea
 - Si hay pocas particiones el clúster queda infrautilizado
- **Mantiene consistencia en los datos**
 - Agrupa eventos relacionados en la misma partición
 - Conserva el orden dentro de cada partición de Kafka
- **La clave debe tener suficiente granularidad**
 - Claves generales concentran demasiada carga en una partición
 - Clave más granulares mejoran el reparto y el rendimiento
 - Ejemplos son `player_id` o `match_id`



- **Rendimiento es un equilibrio entre throughput, latencia y resiliencia**
 - Depende del tamaño del mensaje, compresión y acks
 - Compresión reduce tráfico de red pero añade coste de CPU
 - acks=all y mayor replicación mejoran durabilidad pero aumentan latencia
- **La infraestructura condiciona el resultado**
 - Kafka es intensivo en entrada/salida → conviene usar discos rápidos
 - La red debe ser estable y con suficiente capacidad para picos de tráfico
 - Es buena práctica separar discos de datos y distribuir réplicas entre racks o zonas



- **Número de particiones**

- Aumentan paralelismo y throughput
- Demasiadas añaden overhead

- **Factores de replicación**

- Aumenta resiliencia
- Aumenta el uso de disco y red

- **Retención y compactación del log**

- Retención de mensajes → definir según su uso
- Compactación del log → mantiene último estado por clave



Replicación en Kafka

Esquema conceptual original de la edición abierta

Fuente: <https://developer.confluent.io/courses/architecture/data-replication/>



- **Batching para agrupar mensajes antes de enviar**
 - Definir tiempo de espera para acumular más mensajes
 - Definir tamaño máximo del batch de mensajes (bytes)
 - Ajustes típicos son `linger.ms` y `batch.size`
 - Más throughput a costa de algo más de latencia de envío
- **Compresión de mensajes para reducir tráfico de red y disco**
 - Distintos algoritmos de compresión con diferentes trade-offs
 - Elegir según el tamaño de los mensajes y la CPU disponible
- **Fiabilidad para no perder eventos críticos**
 - Definir el nivel de confirmación de escritura (`acks`)
 - Activar idempotencia para evitar duplicados por reintentos
 - Más fiabilidad suele implicar más latencia de escritura



- **Consumer groups para escalado horizontal**
 - Añadir consumidores al mismo grupo aumenta el paralelismo
 - El máximo útil es 1 consumidor por partición (extra quedan ociosos)
- **Commits de offset para controlar el progreso**
 - Los commits automáticos simplifican la configuración
 - Los commits manuales dan más control sobre cuándo confirmar
- **Reintentos y DLQs para gestionar errores**
 - Definir una política de reintentos antes de descartar mensajes
 - Enviar a una DLQ los eventos que fallan repetidamente
- **Monitorizar retrasos en consumidores para detectar cuellos de botella**



Coordinación de consumidores

Esquema conceptual original de la edición abierta

Fuente: <https://developer.confluent.io/courses/architecture/consumer-group-protocol/>



- **Particiones, tareas y executors están relacionados**
 - Spark divide los datos en particiones lógicas para procesarlos en paralelo
 - Cada partición suele ejecutarse como una tarea independiente
 - Los executors son los procesos que ejecutan esas tareas en los nodos del clúster
- **El shuffle redistribuye datos entre nodos**
 - Es habitual en operaciones como join, groupBy y agregaciones
 - Suele ser una de las fases más costosas por uso de red, disco y tiempo
- **Las particiones impactan en el rendimiento**
 - Si hay pocas particiones se desaprovecha el clúster
 - Si hay demasiadas aumenta la sobrecarga de gestión
 - Si los executors tienen pocos recursos el procesamiento se ralentiza



- **Las particiones marcan el paralelismo**

- Cada partición suele ejecutarse como una tarea por etapa
- Conviene ajustar su número a la capacidad real del clúster
- El objetivo es equilibrar paralelismo y sobrecarga

- **El desbalanceo de particiones es un enemigo del rendimiento**

- Aparece cuando pocas particiones concentran muchos datos (skew)
- Algunas tareas terminan rápido y otras tardan mucho más
- El rendimiento global queda limitado por la partición más grande
- Primero se corrige el skew y después se ajusta el número de particiones

- **Decidir entre repartir o compactar particiones**

- `repartition` → redistribuye datos con *shuffle*
- `coalesce` → reduce particiones con menos coste (normalmente sin *shuffle*)



- **En skew hay tareas que dominan el tiempo total del job**
 - Pocas tareas lentas dominan el job (stragglers)
 - Uso intensivo de disco mientras otros executors quedan ociosos
- **Es clave comparar la duración de tareas en Spark**
 - Si hay grandes diferencias entre tareas, suele haber desbalanceo
 - Revisar también el tamaño de particiones tras joins y agregaciones
- **Existen diferentes técnicas para mitigar el skew**
 - Repartir mejor la carga (claves más granulares o subclaves)
 - Tratar por separado casos con volumen anómalo
 - Usar AQE para rebalanceo automático



Distribución de datos con skew

Esquema conceptual original de la edición abierta



Distribución equilibrada

Esquema conceptual original de la edición abierta

Fuente: <https://blog.scottlogic.com/2018/03/22/apache-spark-performance.html>



- **El shuffle es la operación más costosa**
 - Implica serialización, red y escritura a disco intermedio
 - Suele dominar el tiempo de ejecución en joins y agregaciones
- **Reducir datos antes del shuffle**
 - Filtrar filas y proyectar solo las columnas necesarias
 - Preagregar cuando sea posible antes de un join
 - Menos datos en movimiento implica menos coste y menor latencia
- **Usar broadcast join con tablas pequeñas**
 - Si una tabla cabe en memoria, se copia a los executors
 - Así se evita redistribuir la tabla grande
 - Genera menos shuffle y joins más rápidos



Operación de shuffle

Esquema conceptual original de la edición abierta

Fuente: <https://blog.scottlogic.com/2018/03/22/apache-spark-performance.html>



- **Guarda datos intermedios para reutilizarlos**
 - Solo compensa cuando esos datos se usan varias veces
 - Si un resultado se usa una sola vez suele empeorar el rendimiento
- **La cache ocupa memoria y compite con la ejecución de tareas**
 - Si no hay memoria suficiente aumenta el uso de disco y baja el rendimiento
 - Conviene priorizar datasets intermedios costosos y estables
- **Buenas prácticas de uso**
 - Cachear después de transformaciones costosas que se reutilizan
 - Liberar el cache cuando deja de ser necesario y validar impacto con métricas



- **Persistir extiende la idea de cache**

- El cache es una forma de persistencia en memoria
- La persistencia permite elegir cómo guardar los datos
- Se usa cuando el mismo DataFrame se reutiliza y el volumen puede ser alto

- **Existen varios niveles de persistencia**

- MEMORY_ONLY → más rápido si los datos caben en memoria
- MEMORY_AND_DISK → más robusto cuando el volumen es grande

- **Checkpointing para análisis en streaming**

- Guarda el estado del pipeline para recuperarse tras fallos
- Es clave en ventanas temporales, deduplicación y joins con estado
- Debe guardarse en almacenamiento fiable (HDFS, S3 o equivalente)



- 1 Introducción
- 2 Buenas prácticas
- 3 Puesta en producción**



- **Elegir el modelo de despliegue**

- Bare metal prioriza rendimiento y control de la infraestructura
- Máquinas virtuales ofrecen más flexibilidad con algo de sobrecarga
- Kubernetes facilita la operación pero exige cuidar el almacenamiento persistente

- **Kafka depende mucho del rendimiento de disco y de red**

- Conviene usar almacenamiento rápido para los logs
- También es importante una red estable entre brokers

- **Es fundamental asegurar la continuidad del servicio**

- Se recomiendan al menos 3 brokers y factor de replicación 3 en tópicos críticos
- Distribuir réplicas en distintos nodos o racks mejora la disponibilidad
- KRaft simplifica el despliegue frente a ZooKeeper

- **Dimensionar almacenamiento y límites de mensajes**



- **Conviene elegir el gestor del clúster según el entorno**
 - YARN encaja bien en ecosistemas Hadoop
 - Kubernetes facilita orquestación, aislamiento y elasticidad
 - Spark standalone suele reservarse para desarrollo y pruebas
- **El almacenamiento condiciona el rendimiento y la operación**
 - HDFS es una opción sólida para cargas batch
 - Checkpointing debe guardarse en almacenamiento fiable
- **Es importante separar cargas y fijar dependencias**
 - Conviene separar recursos de batch y streaming
 - Streaming necesita mayor prioridad para mantener la latencia
 - Docker ayuda a mantener dependencias y versiones estables



- **Conviene elegir entre servicio administrado y control directo**
 - Amazon MSK es Kafka administrado por AWS y reduce trabajo de operación
 - MSK Serverless ajusta recursos automáticamente cuando la carga cambia
 - Kafka en EC2 da más control de configuración pero exige más operación
- **La red y el acceso se controlan dentro de AWS**
- **Seguridad y monitorización deben configurarse desde el inicio**
 - IAM puede gestionar autenticación y autorización de clientes
 - CloudWatch centraliza métricas para monitorizar brokers, tópicos y consumo



- **Conviene elegir la plataforma según el tipo de carga**
 - Amazon EMR encaja bien en batch y en streaming
 - Spark sobre EKS aporta flexibilidad y encaje con contenedores
 - Databricks en AWS simplifica desarrollo y operación
- **En AWS Spark se suele lanzar como jobs**
 - Como EMR Steps, spark-submit en EKS o Jobs en Databricks
- **Es importante diferenciar el uso en streaming y en batch**
 - Un flujo habitual conecta MSK o Kinesis con Spark Structured Streaming
 - En streaming se priorizan latencia y estabilidad del pipeline
 - En batch se priorizan throughput y coste por volumen procesado



- **La monitorización debe centrarse en la salud del pipeline**
 - El retraso del consumidor es la métrica principal para detectar retraso
 - También conviene seguir la tasa de producción y consumo por tópico
 - La latencia de extremo a extremo ayuda a comprobar si se cumplen los objetivos
- **La replicación y el disco son claves para la estabilidad**
 - Problemas en el ISR pueden señalar riesgo de pérdida o menor disponibilidad
 - El uso de disco por broker debe vigilarse de forma continua
- **Las alertas y herramientas deben adaptarse al entorno**
 - Configurar alertas por consumer lag, timeouts y errores de broker
 - En on-premises es habitual usar Prometheus y Grafana
 - En AWS se suele usar CloudWatch y métricas de MSK



Observabilidad de Kafka

Esquema conceptual original de la edición abierta

Fuente: <https://grafana.com/blog/get-comprehensive-monitoring-for-your-apache-kafka-ecosystem-instances-quickly-with-grafana-cloud/>



- **Spark Web UI permite localizar cuellos de botella**
 - En la vista de *stages* se ve qué etapas concentran más tiempo
 - Dentro de cada etapa las *tasks* permiten comparar duraciones y detectar skew
 - Las métricas de *shuffle read/write* muestran cuánto dato se mueve entre nodos
- **En streaming hay que vigilar el ritmo del pipeline**
 - La duración del micro-batch debe ser menor que el intervalo del trigger
 - Comparar datos que entran y datos que se procesan ayuda a ver retrasos
 - Los registros pendientes muestran si se está acumulando trabajo
 - Medir tasa de input y procesamiento, duración del batch y watermark
- **Las alertas deben avisar antes de que el pipeline falle**
 - Alertar si aumenta el uso de disco o el tiempo de limpieza de memoria
 - Alertar de inmediato ante jobs fallidos con contexto del error



Interfaz de monitorización de Spark

Esquema conceptual original de la edición abierta

Fuente: <https://spark.apache.org/docs/latest/web-ui.html>



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>